ERD · UML 작업 요청

EMS 명령 이상탐지 MVP 기준 — 테이블 2개, 탐지 규칙 5개, SSE 실시간 흐름

1. ERD

`ems_command_log` — EMS 명령 1건

컬럼	타입	제약
id	BIGINT	PK, auto increment
issued_at	TIMESTAMP	NOT NULL
client_id	VARCHAR(50)	NOT NULL
role	VARCHAR	NOT NULL (OPERATOR · MONITOR · MAINTENANCE · UNKNOWN)
device_id	VARCHAR(50)	NOT NULL
function_code	INT	NOT NULL (3·4 읽기, 6·16 쓰기)
register_address	INT	NOT NULL
command_value	INT	NOT NULL
ess_state	VARCHAR	NOT NULL (STOPPED · RUNNING · MAINTENANCE · FAULT)
labeled_anomaly	BOOLEAN	NOT NULL
processed	BOOLEAN	NOT NULL, 기본 false

인덱스: `(processed, id)`, `(client_id, issued_at)`

`detection_event` — 탐지 결과 1건

컬럼	타입	제약
id	BIGINT	PK, auto increment
command_log_id	BIGINT	NOT NULL
rule_id	VARCHAR(10)	NOT NULL (R1~R5)
severity	VARCHAR	NOT NULL (CRITICAL · HIGH · MEDIUM)
message	VARCHAR(255)	NOT NULL
detected_at	TIMESTAMP	NOT NULL

인덱스: `(detected_at)`, `(command_log_id)`

관계

- `ems_command_log` 1 : N `detection_event` — 명령 하나가 여러 규칙에 걸릴 수 있어요.
- DB FK 제약조건 없이 `command_log_id` 로만 참조하는 **논리적 관계**예요. ERD에는 점선으로 표시하고 "논리 FK"라고 적어 주세요.
- 명령 쪽은 탐지 이벤트가 0개일 수 있어요(정상 명령). 카디널리티는 1 : 0..N.

2. UML

2-1. 클래스 다이어그램

패키지	클래스	표시할 것
command	`CommandLog`, `ClientRole`, `EssState`, `CommandLogRepository`, `CommandLogLoader`, `ReplayService`	`CommandLog`의 `markProcessed()`, `resetProcessed()`, `isWrite()`
detection	`Rule`(인터페이스), `RuleEngine`, `Detection`, `Severity`	`Rule.check(CommandLog): Optional<Detection>`
detection.rule	`UnregisteredClientRule`, `ReadOnlyWriteRule`, `ValueRangeRule`, `StateViolationRule`, `BurstRule`	5개 모두 `Rule` 구현 (실체화 화살표)
event	`DetectionEvent`, `DetectionEventRepository`, `DetectionPublishedEvent`, `DetectionListener`, `EventController`, `EventResponse`, `StatsResponse`	
stream	`SseHub`, `StreamController`	
demo	`ReplayController`	

꼭 들어가야 할 관계

- `RuleEngine` → `Rule`: 1 대 다 (`List<Rule>` 주입)
- `ReplayService` → `CommandLogRepository`, `RuleEngine`, `ApplicationEventPublisher`
- `DetectionListener` → `DetectionEventRepository`, `SseHub`
- `BurstRule` → `CommandLogRepository` (1분 내 쓰기 횟수 조회)
- `DetectionEvent`와 `CommandLog`는 직접 참조하지 않아요. `commandLogId` 필드만 있으니 연관 화살표를 그리지 마세요.

2-2. 시퀀스 다이어그램 (2개)

① 탐지 흐름 — 핵심, 발표용

```
ReplayService (@Scheduled, 1초마다)
 → CommandLogRepository.findFirstByProcessedFalseOrderByIdAsc()
 → RuleEngine.evaluate(commandLog) → 각 Rule.check()
 → [위반 있음] ApplicationEventPublisher.publish(DetectionPublishedEvent)
       → DetectionListener → DetectionEventRepository.save()
                           → SseHub.broadcast() → 대시보드
 → commandLog.markProcessed()
```

② 시연 초기화 흐름

```
대시보드 → ReplayController  POST /api/replay/reset
 → DetectionEventRepository.deleteAll()
 → CommandLogRepository.resetAllProcessed()
 → SseHub.broadcast("reset")
```

3. 산출물과 기준

항목	내용
도구	ERD는 ERDCloud 또는 dbdiagram.io, UML은 draw.io 또는 Mermaid. 팀이 하나로 통일해요.
제출	PNG(발표 자료용) + 원본 파일(수정용)을 저장소 `docs/` 폴더에 올려요.
표기	enum 컬럼은 허용 값을 함께 적어요. 논리 FK는 점선으로 그려요.
확인	엔티티 코드가 확정되면 컬럼명 · 타입이 코드와 같은지 한 번 대조해요.

우선순위: 발표에 쓸 거면 **ERD와 시퀀스** ①이 가장 중요해요. 클래스 다이어그램은 규칙 5개가 하나의 인터페이스를 구현하는 구조("규칙 추가가 쉽다")를 보여주는 데 집중해 주세요.
참고: `evaluate` 같은 메서드 이름은 아직 코드가 없어 예상으로 적었어요. 구현 후 바뀔 수 있으니 코드가 나오면 다이어그램을 한 번 맞춰 주세요.